

Got a problem statement for which we need to build a website for real-life usecase that can be easily used by non technical users, handles realistic values and scenarios & could be easily deployed and without major changes (it's actually for a promptathon).

For now, do not generate the code, just think and evaluate about all the needs, requirements, architecture, design, ui, ux, frontend, backend, data management, security, monitoring etc.

Specially consider that it should work flawlessly, user-friendly & also keep the judges pov in mind that what

Show more ▾



This is actually a much better Promptathon problem than it looks at first glance. Most teams will build a simple flashcard CRUD app with a quiz screen. The winners will likely build a **real study tool** where the Leitner system genuinely influences learning behavior.

1. What Judges Will Probably Evaluate

Even if the rubric isn't given, judges usually look at:

Area	What Average Teams Do	What Strong Teams Do
Functionality	Flashcards + Quiz	Full Leitner scheduling engine
UX	Basic forms	Study-focused workflow
Logic	Fake progress bars	Real mastery calculations
Architecture	One large component	Modular system
Edge Cases	Ignore them	Handle realistically
Deployability	Works locally	Production-ready
Data Persistence	Memory only	Local persistence
Explanation	Features	Educational value

The phrase:

"The scheduling logic must be functional and visible — not cosmetic"

is probably the most important sentence in the entire problem statement.

2. Core Product Vision

Think:

"Could a student preparing for tomorrow's exam actually use this?"

Not:

"Can I technically create flashcards?"

The website should feel like:

- Open website
- Create deck in 2 minutes
- Start studying immediately
- Clearly understand progress
- Understand why certain cards keep reappearing
- Feel rewarded when cards move toward mastery

3. Real Leitner System Understanding

Many participants will misunderstand this.

A real Leitner system contains boxes.

Example:

Box	Meaning	Review Frequency
1	New / Weak	Every round
2	Somewhat known	Less often
3	Good	Even less
4	Strong	Rare

Box	Meaning	Review Frequency
5	Mastered	Very rare

Flow:

Wrong Answer:

Box 3



Box 1

Correct Answer:

Box 1 → Box 2

Box 2 → Box 3

Box 3 → Box 4

Box 4 → Box 5

This is the heart of the challenge.

4. Visible Scheduling Requirement

The problem explicitly demands visibility.

Many teams may implement:

```
card.box = 2
```

internally but never show it.

That likely loses points.

Student should see:

Photosynthesis

Current Box: 2

Next Promotion: Box 3

or

Current Status:

■ Learning

- Improving
- Mastered

Even better:

```
Box 1: 12 cards
Box 2: 8 cards
Box 3: 5 cards
Box 4: 2 cards
Box 5: 1 card
```

Now the algorithm becomes visible.

5. Information Architecture

I would separate the app into 4 major modules.

Module 1: Deck Management

Purpose:

Create flashcards.

Features:

- Add card
- Edit card
- Delete card
- Bulk add
- View deck

Card structure:

```
Question
Answer
Current Leitner Box
Times Correct
Times Wrong
Created Date
Last Reviewed
```

Module 2: Study Session Engine

Purpose:

Run quizzes.

Responsibilities:

- Pick next card
- Show question
- Flip card
- Record answer
- Update Leitner box
- Update score

This is the most important module.

Module 3: Progress Analytics

Purpose:

Show learning progress.

Metrics:

- Total cards
- Remaining cards
- Mastered cards
- Accuracy %
- Current streak
- Box distribution

Module 4: Persistence Layer

Purpose:

Store data.

Could be:

`LocalStorage`

for Promptathon.

No backend required unless explicitly desired.

6. Study Session Design

A very important UX area.

Flow:

Question Screen:

What is Ohm's Law?

[Show Answer]

↓

Answer Revealed:

$V = IR$

Did you get it right?

[ Wrong]

[ Correct]

↓

Immediately show:

Moved to Box 2

or

Moved back to Box 1

Visible feedback reinforces learning.

7. Scheduling Strategy

This is where many submissions become weak.

Do NOT simply review all cards in order.

Example:

Current state:

Box1 = 10 cards
Box2 = 8 cards
Box3 = 6 cards
Box4 = 4 cards

Review priority should favor lower boxes.

Possible weight:

Box1 = 5
Box2 = 3
Box3 = 2
Box4 = 1
Box5 = 0.5

Thus weaker cards appear more frequently.

This satisfies:

incorrectly answered cards must appear more frequently

8. Session Completion Logic

Question:

When does a study session end?

Weak solution:

After all cards seen once

Better solution:

Until no due cards remain

Even better:

Session Target:
20 reviews

or

Study for 15 minutes

But since promptathon scope is limited:

Use

```
Review until due cards exhausted
```

9. Progress Tracking

Required by statement.

Minimum:

```
Cards Remaining  
Cards Mastered  
Session Score
```

But we can elevate it.

Dashboard:

```
Total Cards: 50
```

```
Mastered: 20  
Learning: 30
```

```
Accuracy: 84%
```

```
Current Session:  
Correct: 15  
Wrong: 3
```

10. Data Model

A realistic card should contain:

```
JavaScript  
  
{  
  id,  
  question,  
  answer,  
  
  box: 1,
```



```
    correctCount: 0,  
    wrongCount: 0,  
  
    lastReviewed: null,  
  
    createdAt: timestamp  
  }
```

Simple.

Scalable.

Production-like.

11. UX Considerations

Students want speed.

Card Creation

Bad:

```
Question:  
Answer:  
Save
```

repeated 50 times.

Good:

```
Add Card  
Add Card  
Add Card
```

without page refresh.

Quiz

One-click flow.

```
Question  
↓  
Reveal  
↓  
Correct/Wrong
```

↓
Next

No unnecessary confirmations.

12. Mobile Responsiveness

Judges often test this.

Flashcards naturally fit mobile.

Design should be:

Centered card
Large buttons
Thumb-friendly controls

13. Edge Cases

Strong projects handle these.

Empty Deck

No cards yet.
Create your first flashcard.

One Card Only

Should still work.

All Cards Mastered

Show:

 Congratulations!

All cards are mastered.

Massive Deck

100+ cards.

App should remain responsive.

14. Security

For a Promptathon:

Don't overengineer.

Enough:

Input Sanitization

Prevent:

HTML

`<script>`

in questions.

Validation

Question cannot be empty.

Answer cannot be empty.

Length limits

Example:

Question: 500 chars max

Answer: 1000 chars max

15. Monitoring / Reliability

Even without backend:

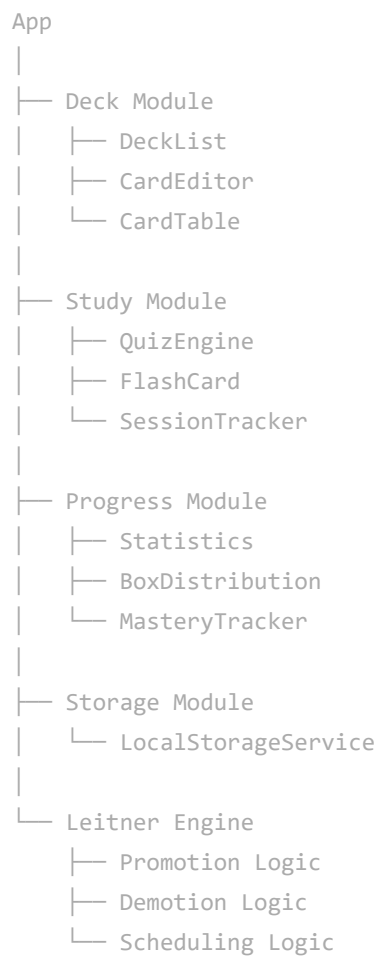
Useful signals:

Total Reviews
Study Time
Cards Mastered Today

These make the product feel real.

16. Architecture Recommendation

For strategic modular programming:



Notice:

The Leitner Engine is isolated.

This is important.

Judges love seeing:

UI Layer
↓
Business Logic Layer



Storage Layer

instead of all logic mixed together.

Biggest Opportunity

Most teams will submit:

Flashcard CRUD + Quiz + Progress Bar

The stronger submission is:

A genuine learning system where students can clearly see cards moving through Leitner boxes, weaker cards return more often, mastery becomes measurable, and the scheduling engine is treated as the centerpiece of the product.

For this problem, I'd estimate the scoring importance roughly as:

1. **Leitner scheduling engine** → 35%
2. **Study session UX** → 25%
3. **Progress visualization** → 15%
4. **Architecture/modularity** → 15%
5. **Polish/responsiveness/validation** → 10%

The Leitner engine is the feature that will most strongly separate top submissions from average ones. 🚀



index(4).html
File

style(1).css
File

i am providing you both html and css completely files,
review and audit everything prepare the changes if
required any!

style(2).css
File

index(5).html
File

script(6).js
JavaScript

style(3).css
File

index(6).html
File

script(7).js
JavaScript

style(4).css
File

index(7).html
File

give me break for a check, review, audit do everything it needs and just get the prompt and then get the part 3 of script.js ready , you can take your time

script(8).js
JavaScript

style(5).css
File

index(8).html
File

do full check we have time for checking only once make
chance count well

script(9).js
JavaScript

style(6).css
File

index(9).html
File

what's actually going on the project is a complete dummy ,
its not working at all

style(7).css
File

index(10).html
File

i am giving you index and style , now lets regenerate the js
this time lets make it work flawlessly

style(8).css
File

index(11).html
File

i would suggest if there's problem then we can regenerate the b part then will move on to c part

script(10).js
JavaScript

style(9).css
File

index(12).html

script(11).js
JavaScript

style(10).css
File

index(13).html
File

we're almost there just need to fix few things like at some place cancel button is not functioning , the bulk adding card feature is halfway there it just not saving the cards, come with solutions and also tell me what's the function of those boxes.

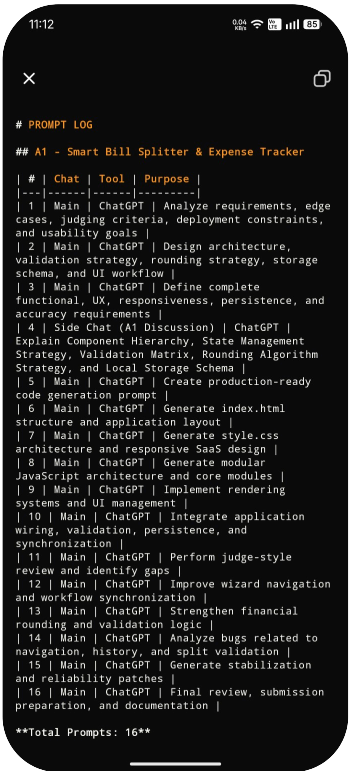
script(12).js
JavaScript

style(11).css
File

index(14).html

File

do a final check



In this format

For the README, keep it simple. Judges usually spend less than 2 minutes reading it.

Use something like this:

Flash Card Study Buddy

Flash Card Study Buddy

A modern flashcard study application built using the Leitner Spaced Repetition System to help students revise efficiently and retain information longer.

Features

- Create multiple study decks
- Add, edit, and delete flashcards

- Bulk import flashcards
- Leitner Box learning system
- Quiz mode with answer evaluation
- Session summary and performance tracking
- Analytics dashboard
- Local storage persistence
- Dark mode support
- Responsive design

Leitner System

Cards are organized into 5 boxes:

Box	Status
Box 1	Learning
Box 2	Improving
Box 3	Familiar
Box 4	Strong
Box 5	Mastered

Promotion Rules

- 1 → 2
- 2 → 3
- 3 → 4
- 4 → 5
- 5 → 5

Demotion Rules

- 1 → 1
- 2 → 1
- 3 → 1
- 4 → 2
- 5 → 3

Cards that are answered correctly move forward, while incorrectly answered cards move backward and appear more frequently in future study sessions.

How To Use

Step 1 – Create a Deck

- Open the application.
- Click **Create Deck**.
- Enter a deck name.
- Save.

Example:

Physics

Chemistry

Biology

Step 2 – Open the Deck

- Click **Open** on the desired deck.

Step 3 – Add Flashcards

Click **Add Card**.

Enter:

Question: What is H₂O?

Answer: Water

Save the card.

Step 4 – Bulk Import Cards (Optional)

Click **Bulk Add Cards**.

Format:

Capital of France | Paris

2 + 2 | 4

Largest Planet | Jupiter

One card per line.

Click **Import Cards**.

Step 5 – Start Studying

Click **Start Quiz**.

The application will show a question.

1. Read the question.
2. Click **Reveal Answer**.
3. Evaluate yourself:
 - Got It Right
 - Got It Wrong

The card will automatically move between Leitner boxes.

Step 6 – Complete Session

When all scheduled cards are reviewed:

- Session Summary appears.
- Review your statistics.
- Study Again if needed.

Step 7 – View Analytics

Open **Analytics**.

Track:

- Total Reviews
- Accuracy
- Mastered Cards
- Box Distribution

Step 8 – Use Dark Mode

Click the **Dark Mode** button in the navigation bar.

Your theme preference is automatically saved.

Data Storage

The application uses browser Local Storage.

All decks, cards, progress, and settings remain available after refreshing the page.

Technologies Used

- HTML5
- CSS3
- JavaScript (ES6)
- Local Storage API

Project Goal

The goal of this project is to provide a simple and effective study tool that combines flashcards with a genuine Leitner Spaced Repetition System to improve long-term retention and exam preparation.

This version explains the app in the same order a first-time user would actually use it, which makes it much easier for judges to understand quickly. 🚀

     Sources

for readme also, give the same feature

If you want the README in the **same style as the Prompt Log screenshot** (short, structured, easy to scan), I'd do something like this:

Flash Card Study Buddy

Overview

A flashcard learning application based on the **Leitner Spaced Repetition System**.

The application helps students revise topics efficiently by showing weaker cards more frequently and stronger cards less frequently.

Features

Feature	Description
Multiple Decks	Create and manage multiple study decks
Flashcards	Add, edit, and delete flashcards
Bulk Import	Import multiple cards at once using Question Answer format
Leitner System	Automatic card promotion and demotion
Quiz Mode	Study cards through interactive sessions
Session Summary	Review performance after each session
Analytics	Track reviews, accuracy, and mastery
Local Storage	Progress automatically saved in browser
Dark Mode	Toggle between light and dark themes
Responsive Design	Works on desktop, tablet, and mobile

Leitner Box System

Box	Status
1	Learning

2	Improving
3	Familiar
4	Strong
5	Mastered

Promotion

- 1 → 2
- 2 → 3
- 3 → 4
- 4 → 5
- 5 → 5

Demotion

- 1 → 1
- 2 → 1
- 3 → 1
- 4 → 2
- 5 → 3

How To Use

Step	Action
1	Create a deck
2	Open the deck
3	Add flashcards or bulk import cards
4	Click Start Quiz
5	Reveal answer
6	Mark Got It Right or Got It Wrong
7	Complete session

Bulk Import Format

Question 1 | Answer 1

Question 2 | Answer 2

Question 3 | Answer 3

One card per line.

Analytics

The Analytics page displays:

- Total Reviews
- Accuracy Percentage
- Mastered Cards
- Box Distribution

Data Persistence

All data is automatically stored using browser Local Storage.

Saved data includes:

- Decks
- Flashcards
- Leitner Progress
- Analytics
- Theme Preference

Technologies Used

Technology

Purpose

HTML5

Structure

CSS3

Styling

JavaScript (ES6)

Application Logic

Local Storage API

Persistence

Project Structure

File

Purpose

index.html

Application Structure

style.css

User Interface & Responsive Design

script.js

Application Logic & State Management

User Flow

Create Deck



Add Cards



Start Quiz



Reveal Answer



Right / Wrong



Leitner Box Update



Session Summary



Analytics

This matches the style of the screenshot much more closely: short sections, tables, minimal paragraphs, and easy for judges to skim in under a minute.

i mean the same markdown format

Got it now 😊

You mean the **README** itself should be in the same table format as the prompt log screenshot, not normal README sections.

Something like:

Markdown

```
# README
```

```
## A2 - Flash Card Study Buddy (Leitner Box System)
```

```
| Feature | Description |
|-----|-----|
| Project Type | Flashcard Study Application |
| Learning Method | Leitner Spaced Repetition System |
| Storage | Browser Local Storage |
| Theme Support | Light Mode + Dark Mode |
| Responsiveness | Mobile, Tablet, Desktop |
| Persistence | Automatic Save & Load |
```

```
---
```

```
## FEATURES
```

```
| Feature | Description |
|-----|-----|
| Deck Management | Create, Rename, Delete, and Open Decks |
| Flashcard Management | Add, Edit, Delete Flashcards |
| Bulk Import | Import multiple flashcards using Question \ Answer format |
| Quiz Mode | Study cards through interactive sessions |
| Leitner System | Automatic promotion and demotion of cards |
| Analytics | Review statistics and box distribution |
| Session Summary | View performance after each study session |
| Dark Mode | Toggle between Light and Dark themes |
```

```
---
```

```
## LEITNER BOX SYSTEM
```

```
| Box | Status |
|-----|-----|
| Box 1 | Learning |
```

	Box 2		Improving	
	Box 3		Familiar	
	Box 4		Strong	
	Box 5		Mastered	

Promotion Rules

	From		To	
	-----		-----	
	1		2	
	2		3	
	3		4	
	4		5	
	5		5	

Demotion Rules

	From		To	
	-----		-----	
	1		1	
	2		1	
	3		1	
	4		2	
	5		3	

HOW TO USE

	Step		Action	
	-----		-----	
	1		Click Create Deck	
	2		Enter Deck Name	
	3		Open the Deck	
	4		Add Cards or Bulk Import Cards	
	5		Click Start Quiz	
	6		Read the Question	
	7		Click Reveal Answer	
	8		Select Got It Right or Got It Wrong	
	9		Continue Until Session Ends	
	10		Review Session Summary	
	11		Check Analytics	

BULK IMPORT FORMAT

	Format	

	Capital of France \	Paris
	Largest Planet \	Jupiter
	2 + 2 \	4

One flashcard per line.

ANALYTICS

Metric	Description
Total Reviews	Number of reviewed flashcards
Accuracy	Correct answer percentage
Mastered Cards	Cards currently in Box 5
Box Distribution	Number of cards in each Leitner Box

TECHNOLOGIES USED

Technology	Purpose
HTML5	Structure
CSS3	Styling
JavaScript (ES6)	Application Logic
Local Storage API	Data Persistence

FILE STRUCTURE

File	Purpose
index.html	Application Structure
style.css	UI Styling and Responsiveness
script.js	Application Logic

USER FLOW

Flow
Create Deck
↓
Add Cards
↓
Start Quiz
↓
Reveal Answer
↓
Right / Wrong
↓
Leitner Box Update

| ↓ |
| Session Summary |
| ↓ |
| Analytics |

This is probably the closest README style to the screenshot while still looking professional for submission. 🚀

